SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED under section 3 of UGC Act 1956)

ENATHUR, KANCHIPURAM – 631 561



OBJECT ORIENTED PROGRAMMING IN JAVA

LABORATORY RECORD

Name : _____

Reg. No : _____

Class : **B.Sc. (CS with Cyber Security)**

Subject : **BSCY233P80 – Object Oriented Programming in Java Lab**

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(University Established under section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by Mr./Ms._____ with Reg.No. _____ of Second Year B.Sc. [Computer Science with Cyber Security] in the Object Oriented Programming in Java Laboratory during the year 2026.

Staff-in-charge **Head of the Department**

Submitted for the Practical Examination held on _____.

Internal Examiner **External Exam**

Index

S.No	Date	Title	Page No	Signature
1.		Java Programming Fundamentals		
2.		Arrays and Strings		
3.		Classes and Objects		
4.		Inheritance and Polymorphism		
5.		Packages and Interfaces		
6.		Exception Handling		
7.		Multithreading		
8.		Introduction to Swing		
9.		Swing Controls		
10.		Event Handling		

Date:

Object Oriented Java Programming Lab Exercises

Exercise 1: Java Program Fundamentals

AIM:

To write a Java Program using **variables, data types, operators, input/output statements, and control structures.**

ALGORITHM:

1) Start the program.
2) Create a **Scanner** object to read input from the user.
3) Declare variables of different data types (**int, double, char, boolean**).
4) Read two integer values from the user.
5) Perform arithmetic operations:
    - Addition
    - Subtraction
    - Multiplication
    - Division (If the second number is not **Zero**).
    - Modulus
6) Calculate the average of the two numbers.
7) Compare the two numbers using a relational operator.
8) Determine the grade using the **if-else** control structure based on the average.
9) Display the results of all operations.
10) Use a **for** loop to display numbers from 1 to 10.
11) Close the **Scanner** object.
12) Stop the program.

SOURCE CODE:

```
Importjava.util.Scanner;
public class Program1 {
public static void main(String[] args) {

    //Scanner Object
    Scanner sc = new Scanner(System.in);

    //Variable Declaration
    int num1, num2;
    double average;
    char grade;
    boolean isGreater;

    //Inputs
    System.out.print("Enter first number: ");
    num1 = sc.nextInt();
    System.out.print("Enter second number: ");
    num2 = sc.nextInt();

    //Operations
    int sum = num1 + num2;
    int difference = num1 - num2;
    int product = num1 * num2;

    double quotient;

    if (num2 != 0) {
        quotient = (double) num1 / num2;
    } else {
        System.out.println("Division by zero is not
allowed.");
        quotient = 0; // Assigning a default value
    }
```

```
        average = (num1 + num2) / 2.0;

        //Relational Operation
        isGreater = num1 > num2;

        //Grade Calculation - IF ELSE
        if (average >= 90) {
            grade = 'A';
        } else if (average >= 80) {
            grade = 'B';
        } else if (average >= 70) {
            grade = 'C';
        } else if (average >= 60) {
            grade = 'D';
        } else {
            grade = 'F';
        }

        //Outputs
        System.out.println("Sum: " + sum);
        System.out.println("Difference: " + difference);
        System.out.println("Product: " + product);
        if (num2 != 0) {
            System.out.println("Quotient: " + quotient);
        } else {
            System.out.println("Quotient: Undefined (division by
zero)");
        }
        System.out.println("Average: " + average);
        System.out.println("Is first number greater than second? "
+ isGreater);
        System.out.println("Grade: " + grade);

        //Loop - For Loop
        System.out.println("Numbers from 1 to 10:");
        for (int i = 1; i <= 10; i++) {
            System.out.print(i + " ");
        }
        System.out.println(); // Print a new line after the
numbers

        //Closing Scanner
        sc.close();
    }
```

```
}
```

COMPILATION PROCESS:

1. Type the java Program in a programming environment and save it with the file name **Program1.java.**
2. Open the Command Prompt or Terminal.
3. Navigate to the folder where the program is saved using the **cd** command.
4. Compile the program using the command: javac Program1.java
5. If no error occurred, then Run the program using the command: java Program1

OUTPUT:

```
PS D:\PROJECTS\Java_Lab_Programs> java Program1/Program1
Enter first number: 18
Enter second number: 2
Sum: 20
Difference: 16
Product: 36
Quotient: 9.0
Average: 10.0
Is first number greater than second? true
Grade: F
Numbers from 1 to 10:
1 2 3 4 5 6 7 8 9 10
```

RESULT:

The Java Program was **compiled and executed successfully.** The Expected output has been generated and verified.

Date:

Exercise 2: Arrays and Strings

AIM:

To develop a Java Program using **one-dimensional array, two-dimensional array, String, and StringBuffer classes.**

ALGORITHM:

1. Start the program.
2. Create a **Scanner** object to read input from the user.
3. Declare a one-dimensional array of size 5.
4. Read five integer elements into the array.
5. Calculate the sum of the array elements.
6. Display the elements and their sum.
7. Declare a two-dimensional (2x2) array.
8. Read the elements of the 2x2 matrix from the user.
9. Display the matrix.
10. Read a string from the user.
11. Perform string operations:
    - Find the length of the string.
    - Convert it to uppercase.
    - Convert it to lowercase.
    - Reverse the string.
12. Create a **StringBuffer** object and Perform the following operations:
    - Append a string.
    - Insert a string.
    - Replace characters.

- Delete characters.
- Reverse the string.

13. Display the results of all operations.
14. Close the **Scanner** object.
15. Stop the program.

SOURCE CODE:

```
import java.util.Scanner;
public class Program2 {
    public static void main(String[] args) {
        //Scanner Object
        Scanner sc = new Scanner(System.in);

        //----- 1D Array -----
        int[] arr = new int[5];
        int sum = 0;
        System.out.println("Enter 5 numbers for the array:");
        for (int i = 0; i < arr.length; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];
        }

        System.out.println("\nElements in the array:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

        System.out.println("\nSum of array elements: " + sum);

        //----- 2D Array -----
        int[][] matrix = new int[2][2];
        System.out.println("\nEnter elements for a 2x2 matrix:");
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }

        System.out.println("\nElements in the 2x2 matrix:");
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
```

```
        //----- String Operations -----
        System.out.print("\nEnter a string: ");
        String inputString = sc.next();
        System.out.println("You entered: " + inputString);
        System.out.println("Length of the string: " +
inputString.length());
        System.out.println("Uppercase: " +
inputString.toUpperCase());
        System.out.println("Lowercase: " +
inputString.toLowerCase());
        System.out.println("Reversed string: " + new
StringBuilder(inputString).reverse());

        //----- StringBuffer Operations -----
        StringBuffer sb = new StringBuffer(inputString);
        sb.append(" Java");
        System.out.println("\nStringBuffer after appending: " +
sb);

        sb.insert(0, "Hello ");
        System.out.println("StringBuffer after inserting: " + sb);

        sb.delete(0, 6);
        System.out.println("StringBuffer after deleting: " + sb);

        sb.replace(0, 4, "Hi");
        System.out.println("StringBuffer after replacing: " + sb);

        sb.reverse();
        System.out.println("StringBuffer after reversing: " + sb);

        sc.close();
    }
}
```

COMPILATION PROCESS:

1. Type the java Program in a programming environment and save it with the file name **Program2.java.**
2. Open the Command Prompt or Terminal.
3. Navigate to the folder where the program is saved using the **cd** command.
4. Compile the program using the command: javac Program2.java
5. If no error occurred, then Run the program using the command: java Program2

OUTPUT:

```
PS D:\PROJECTS\Java_Lab_Programs> java Program2/Program2
Enter 5 numbers for the array:
1 2 3 4 5

Elements in the array:
1 2 3 4 5
Sum of array elements: 15

Enter elements for a 2x2 matrix:
1 2 3 4

Elements in the 2x2 matrix:
1 2
3 4

Enter a string: Hello
You entered: Hello
Length of the string: 5
Uppercase: HELLO
Lowercase: hello
Reversed string: olleH

StringBuffer after appending: Hello Java
StringBuffer after inserting: Hello Hello Java
StringBuffer after deleting: Hello Java
StringBuffer after replacing: Hio Java
StringBuffer after reversing: avaJ oiH
```

RESULT:

The Java Program was **compiled and executed successfully.** The Expected output has been generated and verified.

Date:

Exercise 3: Classes and Object

AIM:

To develop a Java Program demonstrating **Classes, Objects, Methods, and Constructors**

ALGORITHM:

1. Start the program.
2. Import the **Scanner** class for the user input.
3. Define a **Student** class with data members:
    - Roll Number
    - Name
    - Marks array
    - Total and Average
    - Grade
4. Create a parameterized constructor to initialize the student's roll number and name.
5. Create the **getMarks()** method to
    - Read marks for 5 subjects.
    - Store them in an array.
    - Calculate the total marks.
    - Calculate the average marks.
6. Create the **calculateGrade()** method to determine the grade based on the average.
7. Create the **displayDetails()** method to display the student's roll number, name, subject marks, total, average, and grade.
8. In the **main()** method:

- Read the student's roll number and name.
- Create a **student** object using the constructor.
- Call the **getMarks()** method.
- Call the **calculateGrade()** method.
- Call the **displayDetails()** method.

9. Close the **Scanner** object.
10. Stop the program.

SOURCE CODE:

```
import java.util.Scanner;

class Student {
    int rollNo;
    String name;
    int[] marks = new int[5];
    int total;
    double average;
    char grade;

    Student(int rollNo, String name){
        this.rollNo = rollNo;
        this.name = name;
    }

    //methods to get marks
    void getMarks(Scanner sc){
        total = 0;
        System.out.println("\nEnter marks for 5 subjects: ");
        for(int i=0; i<5; i++){
            System.out.println("Subject " + (i+1) + ": ");
            marks[i] = sc.nextInt();
            total += marks[i];
        }
        average = (double)total / marks.length;
    }

    //calculate grade
    void calculateGrade(){
        if(average >= 90){
            grade = 'A';
        } else if(average >= 80){
            grade = 'B';
```

```
        } else if(average >= 70){
            grade = 'C';
        } else if(average >= 60){
            grade = 'D';
        } else {
            grade = 'F';
        }
    }

    //display student details
    void displayDetails(){
        System.out.println("\nStudent Details:");
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
        System.out.println("Marks: ");
        for(int i=0; i<marks.length; i++){
            System.out.println("Subject " + (i+1) + ": " +
marks[i]);
        }
        System.out.println("Total Marks: " + total);
        System.out.println("Average Marks: " + average);
        System.out.println("Grade: " + grade);
    }
}

public class Program3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter roll number: ");
        int rollNo = sc.nextInt();
        sc.nextLine(); // consume the newline character
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        Student student = new Student(rollNo, name);
        student.getMarks(sc);
        student.calculateGrade();
        student.displayDetails();
        sc.close();
    }
}
```

COMPILATION STEPS:

1. Type the java Program in a programming environment and save it with the file name **Program3.java.**
2. Open the Command Prompt or Terminal.
3. Navigate to the folder where the program is saved using the **cd** command.
4. Compile the program using the command: javac Program3.java
5. If no error occurred, then Run the program using the command: java Program3

OUTPUT:

```
PS D:\PROJECTS\Java_Lab_Programs> java Program3/Program3
Enter roll number: 101
Enter name: Sathya

Enter marks for 5 subjects:
Subject 1:
90
Subject 2:
93
Subject 3:
83
Subject 4:
85
Subject 5:
92

Student Details:
Roll No: 101
Name: Sathya
Marks:
Subject 1: 90
Subject 2: 93
Subject 3: 83
Subject 4: 85
Subject 5: 92
Total Marks: 443
Average Marks: 88.6
Grade: B
```

RESULT:

The Java Program was **compiled and executed successfully.** The Expected output as been generated and verified.

Date:

Exercise 4: Inheritance and Polymorphism

AIM:

To develop a Java Program demonstrating the concepts of **Single Inheritance, Multilevel Inheritance, Hierarchical Inheritance, Method Overloading, and Method Overriding.**

ALGORITHM:

1. Start the program.
2. Create a superclass named **Person** with a data member **name** and a method **display()**.
3. Create a subclass **Student** that inherits from **Person.**
4. Override the **display()** method in the **Student** class.
5. Create two overloaded **showDetails()** methods in the **Student** class.
6. Create another subclass **SportsStudent** that inherits from **Student** to demonstrates Multilevel Inheritance.
7. Override the **display()** method in the **SportsStudent** class.
8. Create another subclass **Teacher** that inherits from **Person** to demonstrate Hierarchical Inheritance.
9. Override the **display()** method in the **Teacher** class.
10. In the **main()** method, create objects of the **Student, SportsStudent,** and **Teacher** classes.
11. Call the appropriate methods to demonstrate Inheritance, Method Overloading, and Method Overriding.
12. Display the output.
13. Stop the Program.

SOURCE CODE:

```
class Person{
    String name;

    Person(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("Name: " + name);
    }
}

//Single Inheritance
class Student extends Person {
    int rollNo;

    Student(String name, int rollNo) {
        super(name);
        this.rollNo = rollNo;
    }

    //Method Overriding
    @Override
    void display() {
        super.display();
        System.out.println("Roll No: " + rollNo);
    }

    //Method Overloading
    void showDetails(){
        System.out.println("Student details displayed
successfully.");
    }

    void showDetails(String dept){
        System.out.println("Department: " + dept);
    }
}

//Multilevel Inheritance
class SportsStudent extends Student {
```

```
    String sport;

    SportsStudent(String name, int rollNo, String sport) {
        super(name, rollNo);
        this.sport = sport;
    }

    @Override
    void display() {
        super.display();
        System.out.println("Sport: " + sport);
    }
}

//Hierarchical Inheritance
class Teacher extends Person {
    String subject;
    Teacher(String name, String subject) {
        super(name);
        this.subject = subject;
    }

    @Override
    void display() {
        super.display();
        System.out.println("Subject: " + subject);
    }
}

public class Program4 {
    public static void main(String[] args) {
        System.out.println("===== Student ======");
        Student student = new Student("John Doe", 101);
        student.display();
        student.showDetails();
        student.showDetails("Computer Science");

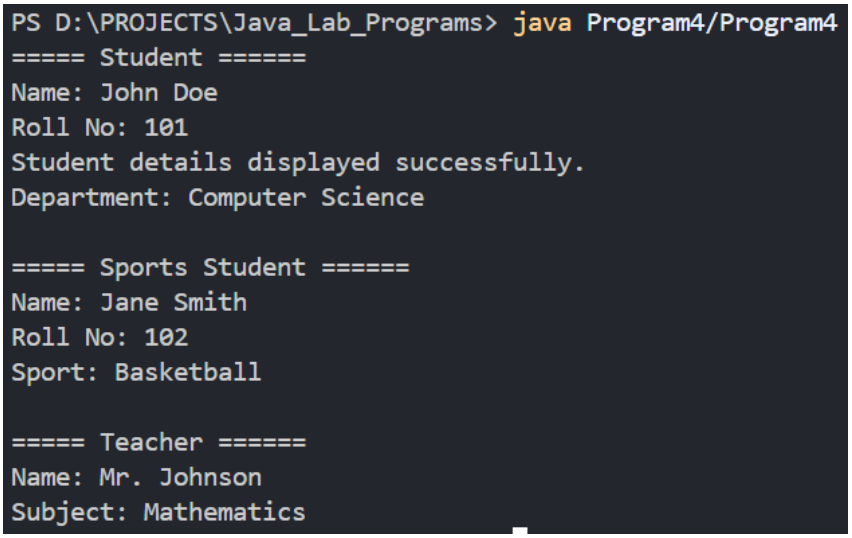
        System.out.println("\n===== Sports Student ======");
        SportsStudent sportsStudent = new SportsStudent("Jane
Smith", 102, "Basketball");
        sportsStudent.display();

        System.out.println("\n===== Teacher ======");
        Teacher teacher = new Teacher("Mr. Johnson",
"Mathematics");
        teacher.display();
    }
}
```

COMPILATION STEPS:

1. Type the java Program in a programming environment and save it with the file name **Program4.java.**
2. Open the Command Prompt or Terminal.
3. Navigate to the folder where the program is saved using the **cd** command.
4. Compile the program using the command: javac Program4.java
5. If no error occurred, then Run the program using the command: java Program4

OUTPUT:



RESULT:

The Java Program was **compiled and executed successfully.** The Expected output has been generated and verified.

Date:

Exercise 5: Packages and Interfaces

AIM:

To develop a Java Program using an **Interface** and a **User-defined Package** to perform simple Arithmetic operations.

ALGORITHM:

1. Start the program.
2. Create an interface **Calculator** inside a package named **mypackage.**
3. Declare methods in the interface:
   - **add()**
   - **subtract()**
   - **product()**
   - **divide()**
   - **modulus()**
4. Import the **Calculator** interface into the main program.
5. Create a class **MyCalculator** that implements the **Calculator** interface.
6. Override the methods inside the class **MyCalculator.**
7. In the main method:
   - Declare a **Scanner** object and read two integer values from user.
   - Create an object of the class **MyCalculator**
   - Call the **Overridden** methods using the class object to perform the operations.
8. Display the results.
9. Stop the program.

STRUCTURE:

```
Program_Folder/
 |
 |- Exercise4.java
 |- MyPackage/
      |
      |- Calculator.java
```

SOURCE CODE:

Program_Folder/MyPackage/Calculator.java:

```
package MyPackage;

public interface Calculator {
    public int add(int a, int b);
    public int subtract(int a, int b);
    public int multiply(int a, int b);
    public double divide(int a, int b);
    public int modulus(int a, int b);
}
```

Program_Folder/Program5.java:

```
import MyPackage.Calculator;
import java.util.Scanner;
class MyCalculator implements Calculator {
    @Override
    public int add(int a, int b) {
        return a + b;
    }

    @Override
    public int subtract(int a, int b) {
        return a - b;
    }

    @Override
    public int multiply(int a, int b) {
        return a * b;
    }

    @Override
    public double divide(int a, int b) {
        if (b == 0) {
            System.out.println("Error: Division by zero is not
allowed.");
```

```
        }
        return (double) a / b;
    }

    @Override
    public int modulus(int a, int b) {
        if (b == 0) {
            System.out.println("Error: Modulus by zero is not
allowed.");
            return 0; // Return 0 or handle the error as needed
        } else {
            return a % b;
        }
    }
}

public class Program5 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int num1 = sc.nextInt();
        System.out.print("Enter second number: ");
        int num2 = sc.nextInt();

        MyCalculator calculator = new MyCalculator();
        System.out.println("Addition: " + calculator.add(num1,
num2));
        System.out.println("Subtraction: " +
calculator.subtract(num1, num2));
        System.out.println("Multiplication: " +
calculator.multiply(num1, num2));
        if(num2 != 0) {
            System.out.println("Division: " +
calculator.divide(num1, num2));
        } else {
            System.out.println("Division: Error: Division by zero
is not allowed.");
        }

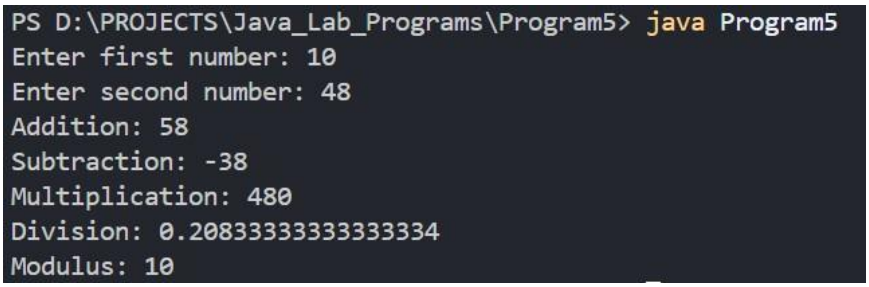
        if(num2 != 0) {
            System.out.println("Modulus: " +
calculator.modulus(num1, num2));
        } else {
            System.out.println("Modulus: Error: Modulus by zero is
not allowed.");
        }

        sc.close();
    }
}
```

COMPILATION STEPS:

1. Type the java Program in a programming environment and save it with the file name **Program5.java.**
2. Open the Command Prompt or Terminal.
3. First change the directory to the Package **MyPackage** using the **cd**, and compile the **Calculator.java** by using the command: javac Calculator.java.
4. Then Change back to the parent directory, and compile the file **Program5.java** using the command: **javac Program5.java**
5. Execute the code Exercise5.java using the command: **java Program5.**

OUTPUT:



RESULT:

The Java Program was **compiled and executed successfully.** The Expected output has been generated and verified.

Date:

Program-6: Exception Handling

Aim:

To write a Java Program demonstrating exception handling using *try, catch, finally, throw, throws, and a user-defined exception.*

Algorithm:

1. Start the Program.
2. Create a user-defined exception class named AgeException by extending the Exception class
3. Create a method checkage() that uses the throws keyword to declare AgeException.
4. Inside checkage(), check whether the given age is less that 18.
5. If the age is less than 18, use throw to generate an AgeException.
6. Otherwise, display that the person is eligible.
7. In the main() method, use a try block to call checkage().
8. Use a catch block to handle the user-defined exception.
9. Add another try-catch block to demonstrate handling of an arithmetic exception.
10. Use a finally block to display message that the block is always executed.
11. Stop the Program.

Source Code:

```
import java.util.Scanner;
// User-defined exception
class AgeException extends Exception {
   AgeException(String message) {
      super(message);
   }
}

public class Program6 {

   // Method using throws
   static void checkAge(int age) throws AgeException {
      if (age < 18) {
```

```
            // Using throw
            throw new AgeException("Age must be 18 or above.");
        } else {
            System.out.println("Eligible to vote.");
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // try-catch-finally with user-defined exception
        try {
            System.out.print("Enter your age: ");
            int age = sc.nextInt();

            checkAge(age);
        }
        catch (AgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
        finally {
            System.out.println("Age verification completed.");
        }

        // try-catch for built-in exception
        try {
            System.out.print("Enter a number: ");
            int num = sc.nextInt();

            int result = 100 / num;
            System.out.println("Result = " + result);
        }
        catch (ArithmeticException e) {
            System.out.println("Exception: Cannot divide by zero.");
        }
        finally {
            System.out.println("Arithmetic operation completed.");
        }
        sc.close();
    }
}
```

Compilation Steps:

1. Type the java Program in a programming environment and save it with the file name Program6.java.
2. Open the Command Prompt or Terminal
3. Navigate to the folder where the program is saved using the cd command.
4. Compile the program using the command: javac Program6.java
5. If no error occurred, then Run the program using the command: java Program6.

Output:

```
PS D:\PROJECTS\Java_Lab_Programs\Program6> javac Program6.java
PS D:\PROJECTS\Java_Lab_Programs\Program6> java Program6
Enter your age: 19
Eligible to vote.
Age verification completed.
Enter a number: 20
Result = 5
Arithmetic operation completed.
PS D:\PROJECTS\Java_Lab_Programs\Program6> java Program6
Enter your age: 17
Exception: Age must be 18 or above.
Age verification completed.
Enter a number: 28
Result = 3
Arithmetic operation completed.
PS D:\PROJECTS\Java_Lab_Programs\Program6> javac Program6.java
PS D:\PROJECTS\Java_Lab_Programs\Program6> java Program6
Enter your age: 18
Eligible to vote.
Age verification completed.
Enter a number: 0
Exception: Cannot divide by zero.
Arithmetic operation completed.
```

Result:

Thus, the Java Program was successfully executed to demonstrate try, catch, finally, throw and throws, and user-defined exception handling.

Date:

Program-7: Multithreading

Aim:

To write a Java Program to create and execute multiple threads using the **Thread class** and **Runnable interface.**

Algorithm:

1. Start the program.
2. Create a class **MyThread** by extending the **Thread** class**.**
3. Override the **run()** method in **MyThread**.
4. Create another class **MyRunnable** that implements the **Runnable** interface.
5. Override the **run()** method in **MyRunnable**.
6. Create objects of both classes.
7. Create a **Thread** object using the **MyRunnable** object.
8. Start both threads using the **start()** method.
9. The **run()** methods execute concurrently.
10. Display the thread name and its iteration number.
11. Use **join()** to wait for both threads to complete.
12. Display a message indicating that all threads have completed.
13. Stop the Program.

Source Code:

```
// Program to demonstrate Multithreading

// Thread class
class MyThread extends Thread {

   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println("Thread Class: " + i);

         try {
            Thread.sleep(500);
         } catch (InterruptedException e) {
            System.out.println("Thread interrupted.");
         }
      }
```

```
    }
}

// Runnable interface
class MyRunnable implements Runnable {

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable Interface: " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted.");
            }
        }
    }
}

public class Program7 {

    public static void main(String[] args) {

        // Creating thread using Thread class
        MyThread t1 = new MyThread();

        // Creating thread using Runnable interface
        MyRunnable r = new MyRunnable();
        Thread t2 = new Thread(r);

        // Starting both threads
        t1.start();
        t2.start();

        try {
            // Wait for both threads to finish
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }

        System.out.println("All threads completed.");
    }
```

Compilation Steps:

1. Type the java Program in a programming environment and save it with the file name Program7.java.
2. Open the Command Prompt or Terminal
3. Navigate to the folder where the program is saved using the cd command.
4. Compile the program using the command: javac Program7.java
5. If no error occurred, then Run the program using the command: java Program7.

Output:

```
PS D:\PROJECTS\Java_Lab_Programs\Program6> cd ../Program7
PS D:\PROJECTS\Java_Lab_Programs\Program7> javac Program7.java
PS D:\PROJECTS\Java_Lab_Programs\Program7> java Program7
Runnable Interface: 1
Thread Class: 1
Thread Class: 2
Runnable Interface: 2
Runnable Interface: 3
Thread Class: 3
Runnable Interface: 4
Thread Class: 4
Thread Class: 5
Runnable Interface: 5
All threads completed.
```

Result:

Thus, the Java Program was successfully executed to demonstrate multithreading using the Thread class and Runnable interface.

Date:

Program-8: Introduction to Swing

Aim:

To write a Java Swing program to create a basic GUI application using **JFrame**, **JPanel**, **JLabel**, **JButton**, **JTextField**, **JTextArea**.

Algorithm:

1. Start the Program.
2. Create a class named **SwingDemo** that extends **JFrame**.
3. Create a **JPanel** to hold the GUI Components.
4. Create a **JLabel** components to display labels.
5. Create a **JTextField** to accept a name from the user.
6. Create a **JTextArea** to display the entered information.
7. Create a **JButton** named Submit.
8. Add an **ActionListener** to the button.
9. When the button is clicked, read the name form the text field.
10. Display the entered name in the text area.
11. Add all components to the panel.
12. Add the panel to the frame.
13. Set the frame size and make it visible.
14. Stop the program.

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Program8 extends JFrame {

   JLabel nameLabel;
   JTextField nameField;
   JButton submitButton;
   JTextArea outputArea;
   JPanel panel;

   Program8() {
```

```
        // Create panel
        panel = new JPanel();
        panel.setLayout(new FlowLayout());

        // Create label
        nameLabel = new JLabel("Enter Your Name:");

        // Create text field
        nameField = new JTextField(15);

        // Create button
        submitButton = new JButton("Submit");

        // Create text area
        outputArea = new JTextArea(5, 25);
        outputArea.setEditable(false);

        // Add action to button
        submitButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {

                String name = nameField.getText();

                outputArea.setText("Hello, " + name + "!\n" + "Welcome to Java Swing.");
            }
        });

        // Add components to panel
        panel.add(nameLabel);
        panel.add(nameField);
        panel.add(submitButton);
        panel.add(outputArea);

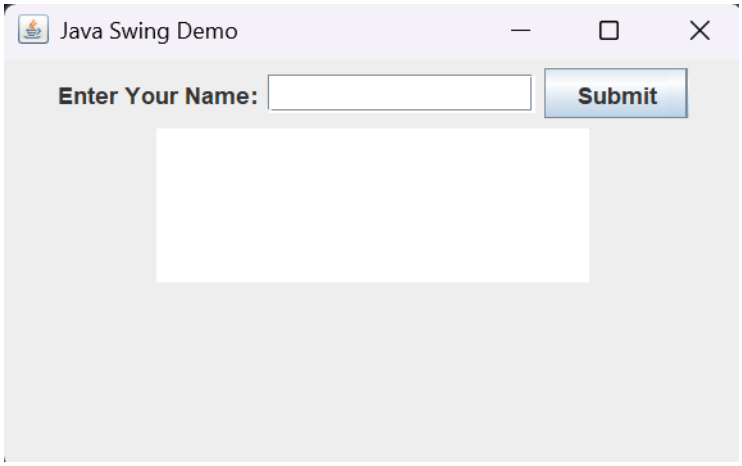
        // Add panel to frame
        add(panel);

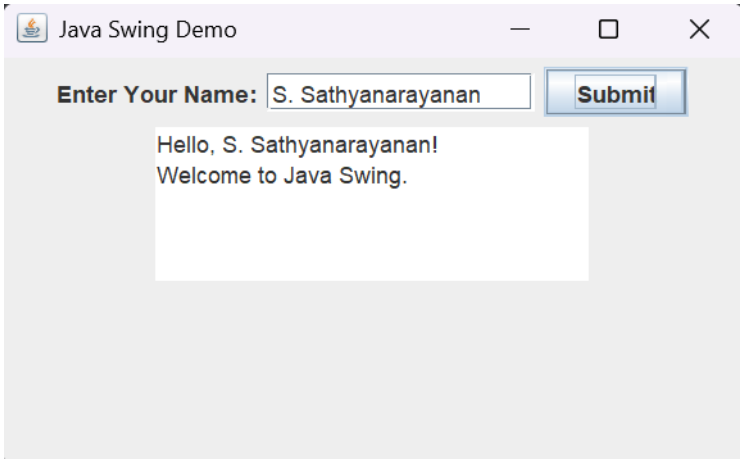
        // Frame settings
        setTitle("Java Swing Demo");
        setSize(400, 250);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
    public static void main(String[] args) {
        new Program8();
    }
}
```

Compilation Steps:

1. Type the java Program in a programming environment and save it with the file name Program8.java.
2. Open the Command Prompt or Terminal
3. Navigate to the folder where the program is saved using the cd command.
4. Compile the program using the command: javac Program8.java
5. If no error occurred, then Run the program using the command: java Program8.

Output:





Result:

Thus, the Java Swing program was successfully executed to create a basic GUI application using **JFrame, JPanel, JLabel, JButton, JTextField, and JTextArea**.

Date:

Program-9: Swing Controls

Aim:

To write a Java Swing program to demonstrate various Swing controls such as **JCheckBox, JRadioButton, JComboBox, JList, JTable, JScrollPane, and JOptionPane**.

Algorithm:

1. Start the program.
2. Create a JFrame to display the GUI.
3. Create a JCheckBox for selecting a programming language.
4. Create JRadioButton components for selecting gender.
5. Group the radio buttons using ButtonGroup.
6. Create a JComboBox for selecting a course.
7. Create a JList for selecting skills.
8. Create a JTable to display student details.
9. Add the JTbale inside a JScrollPane so the table can be scrolled.
10. Create a JButton named Submit.
11. When the Submit button is clicked, retrieve the selected values.
12. Display the selectd information using JOptionPane.
13. Set the frame size and make it visible.
14. Stop the program.

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Program9 extends JFrame {

   JCheckBox javaCheckBox;
   JRadioButton maleRadio;
   JRadioButton femaleRadio;
   JComboBox<String> courseComboBox;
   JList<String> skillsList;
   JTable studentTable;
   JButton submitButton;

   Program9() {
```

```
setTitle("Swing Controls Demo");
setSize(600, 600);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

JPanel panel = new JPanel();
panel.setLayout(new FlowLayout());

// JCheckBox
javaCheckBox = new JCheckBox("Java");

// JRadioButton
maleRadio = new JRadioButton("Male");
femaleRadio = new JRadioButton("Female");

ButtonGroup genderGroup = new ButtonGroup();
genderGroup.add(maleRadio);
genderGroup.add(femaleRadio);

// JComboBox
String[] courses = {
   "B.Sc Cybersecurity",
   "BCA",
   "B.Sc Computer Science",
   "B.Tech IT"
};

courseComboBox = new JComboBox<>(courses);

// JList
String[] skills = {
   "Java",
   "Python",
   "HTML",
   "CSS",
   "JavaScript"
};

skillsList = new JList<>(skills);
skillsList.setVisibleRowCount(3);

JScrollPane listScrollPane = new JScrollPane(skillsList);

// JTable
String[] columns = {"ID", "Name", "Course"};

String[][] data = {
```

```
    {"101", "Arun", "Cybersecurity"},
    {"102", "Bala", "BCA"},
    {"103", "Kiran", "Computer Science"}
};

studentTable = new JTable(data, columns);

JScrollPane tableScrollPane = new JScrollPane(studentTable);

// Submit button
submitButton = new JButton("Submit");

submitButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        String gender = "";

        if (maleRadio.isSelected()) {
            gender = "Male";
        } else if (femaleRadio.isSelected()) {
            gender = "Female";
        } else {
            gender = "Not Selected";
        }

        String course =
                courseComboBox.getSelectedItem().toString();

        String language;

        if (javaCheckBox.isSelected()) {
            language = "Java";
        } else {
            language = "Not Selected";
        }

        String skillsSelected =
                skillsList.getSelectedValue();

        if (skillsSelected == null) {
            skillsSelected = "Not Selected";
        }

        String message =
                "Gender: " + gender +
                "\nCourse: " + course +
                "\nLanguage: " + language +
```

```
                "\nSkill: " + skillsSelected;

            JOptionPane.showMessageDialog(
                Program9.this,
                message,
                "Selected Information",
                JOptionPane.INFORMATION_MESSAGE
            );
        }
    });

    // Add components
    panel.add(new JLabel("Select Language:"));
    panel.add(javaCheckBox);

    panel.add(new JLabel("Gender:"));
    panel.add(maleRadio);
    panel.add(femaleRadio);

    panel.add(new JLabel("Course:"));
    panel.add(courseComboBox);

    panel.add(new JLabel("Select Skill:"));
    panel.add(listScrollPane);

    panel.add(tableScrollPane);

    panel.add(submitButton);

    add(panel);

    setVisible(true);
  }


  public static void main(String[] args) {

    new Program9();

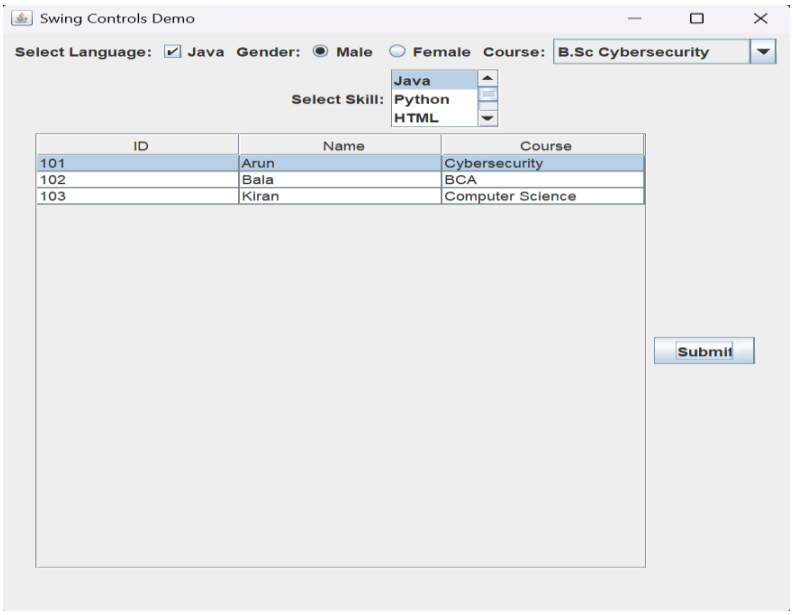
  }

}
```

Compilation Steps:

1. Type the java Program in a programming environment and save it with the file name Program9.java.
2. Open the Command Prompt or Terminal
3. Navigate to the folder where the program is saved using the cd command.
4. Compile the program using the command: javac Program9.java
5. If no error occurred, then Run the program using the command: java Program9.

Output:





Result:

Thus, the Java Swing program was successfully executed to demonstrate **JCheckBox, JRadioButton, JComboBox, JList, JTable, JScrollPane, and JOptionPane.**

Date:

Program-10: Event Handling

Aim:

To write a Java Swing program to demonstrate event handling using ActionListener, ItemListener, MouseListener, MouseMotionListener, KeyListener, and WindowListener.

Algorithm:

1. Start the Program.
2. Create a JFrame with required Swing components.
3. Create a JButton and register an ActionListener to handle button clicks.
4. Create a JCheckBox and register an ItemListener to detect selection changes.
5. Register a MouseListener with the frame to detect mouse clicks and other mouse events.
6. Register a MouseMotionListener to detect mouse movement.
7. Create a JTextField and register a KeyListener to detect keyboard events.
8. Register a WindowListener with the frame to handle window events.
9. Display the event information in a JTextArea.
10. Perform different actions such as clicking the button, selecting the checkbox, moving the mouse, pressing keys, and closing the window.
11. Display the corresponding event messages.
12. Stop the program.

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Program10 extends JFrame implements
      ActionListener, ItemListener, MouseListener,
      MouseMotionListener, KeyListener,
      WindowListener {

   JButton button;
   JCheckBox checkBox;
   JTextField textField;
   JTextArea textArea;
```

```
Program10() {

   setTitle("Java Event Handling");
   setSize(600, 500);
   setLayout(new FlowLayout());

   // Create components
   button = new JButton("Click Me");
   checkBox = new JCheckBox("Select Me");
   textField = new JTextField(20);
   textArea = new JTextArea(15, 45);

   textArea.setEditable(false);

   // Register listeners
   button.addActionListener(this);
   checkBox.addItemListener(this);
   addMouseListener(this);
   addMouseMotionListener(this);
   textField.addKeyListener(this);
   addWindowListener(this);

   // Add components
   add(button);
   add(checkBox);

   add(new JLabel("Type something:"));
   add(textField);

   add(new JScrollPane(textArea));

   setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
   setVisible(true);
}

// ActionListener
public void actionPerformed(ActionEvent e) {
   textArea.append("ActionListener: Button clicked\n");
}

// ItemListener
public void itemStateChanged(ItemEvent e) {

   if (checkBox.isSelected()) {
      textArea.append("ItemListener: Checkbox selected\n");
   } else {
```

```
        textArea.append("ItemListener: Checkbox deselected\n");
    }

    // MouseListener
    public void mouseClicked(MouseEvent e) {
        textArea.append("MouseListener: Mouse clicked\n");
    }

    public void mousePressed(MouseEvent e) {
        textArea.append("MouseListener: Mouse pressed\n");
    }

    public void mouseReleased(MouseEvent e) {
        textArea.append("MouseListener: Mouse released\n");
    }

    public void mouseEntered(MouseEvent e) {
        textArea.append("MouseListener: Mouse entered window\n");
    }

    public void mouseExited(MouseEvent e) {
        textArea.append("MouseListener: Mouse exited window\n");
    }

    // MouseMotionListener
    public void mouseMoved(MouseEvent e) {
        textArea.append("MouseMotionListener: Mouse moved\n");
    }

    public void mouseDragged(MouseEvent e) {
        textArea.append("MouseMotionListener: Mouse dragged\n");
    }

    // KeyListener
    public void keyTyped(KeyEvent e) {
        textArea.append("KeyListener: Key typed = " + e.getKeyChar() + "\n");
    }

    public void keyPressed(KeyEvent e) {
        textArea.append("KeyListener: Key pressed\n");
    }

    public void keyReleased(KeyEvent e) {
        textArea.append("KeyListener: Key released\n");
    }
```

```
    // WindowListener

    public void windowOpened(WindowEvent e) {
        textArea.append("WindowListener: Window opened\n");
    }

    public void windowClosing(WindowEvent e) {
        textArea.append("WindowListener: Window closing\n");
        JOptionPane.showMessageDialog(
                this,
                "Window is closing."
        );
        dispose();
    }

    public void windowClosed(WindowEvent e) {
        System.out.println("WindowListener: Window closed");
    }

    public void windowIconified(WindowEvent e) {
        textArea.append("WindowListener: Window minimized\n");
    }

    public void windowDeiconified(WindowEvent e) {
        textArea.append("WindowListener: Window restored\n");
    }

    public void windowActivated(WindowEvent e) {
        textArea.append("WindowListener: Window activated\n");
    }

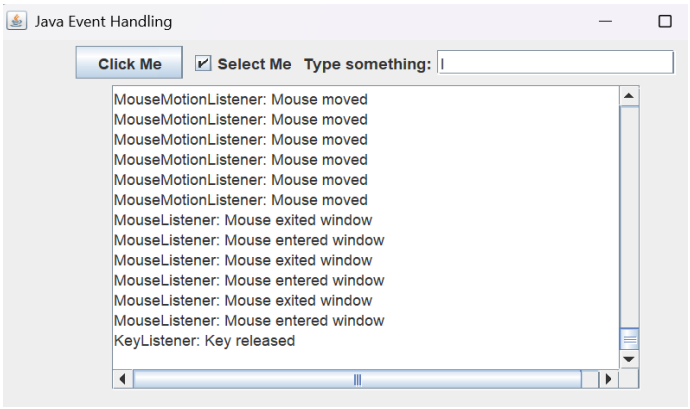
    public void windowDeactivated(WindowEvent e) {
        textArea.append("WindowListener: Window deactivated\n");
    }

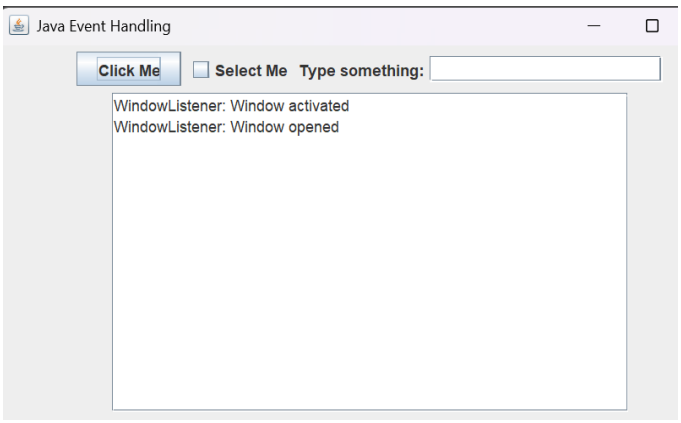
    public static void main(String[] args) {
        new Program10();
    }
}
```

Compilation Steps:

1. Type the java Program in a programming environment and save it with the file name Program10.java.
2. Open the Command Prompt or Terminal
3. Navigate to the folder where the program is saved using the cd command.
4. Compile the program using the command: javac Program10.java
5. If no error occurred, then Run the program using the command: java Program10.

Output:





Result:

Thus, the Java Swing program was successfully executed to demonstrate event handling using **ActionListener, ItemListener, MouseListener, MouseMotionListener, KeyListener, and WindowListener**.